Retransmit, Backfill or Reconstruct? Loss Handling for River Water-Quality Telemetry under Real Outages

Kofi Sarpong Adu-Manu and Stephen Kwaku Azongo

***Abstract*—Battery-powered river water-quality nodes lose data, yet the usual remedies assume short, independent losses. This paper measures loss in four real systems: two water-quality deployments (Kosovo and Ghana) and two LoRaWAN networks (London and Virginia). Losses form two regimes: short radio losses and rare but long outages of the gateway, backhaul or server. Outages of a day or more hold 50.7% of the lost readings in one deployment and 71.0%–100.0% across the streams of the other, and the memoryless loss model fits worst on every record with outages. Replaying the records through a sourced LoRaWAN and NB-IoT energy model shows that retransmission (ARQ) costs 2.59×–7.35× the energy of unconfirmed reporting on LoRaWAN and recovers none of the outage losses, whereas buffering with back-off delivers every reading at a modest energy cost. Reconstruction cannot replace delivery: even the best estimator keeps a median of only 26% of day-long gaps within tolerance. A reporting design combining hourly batching, immediate event uploads, a bounded-error trigger and buffering with back-off was evaluated on 30 river sites against pre-specified hypotheses. With the link up, it raises alarms as fast as per-reading reporting and matches its real-time picture within 1 percentage point, while per-reading ARQ uses 1.30×–2.32× as much energy.**

***Index Terms*—Packet loss, network outages, store-and-forward, ARQ, LoRaWAN, NB-IoT, water-quality monitoring, data reconstruction, energy efficiency, event detection.**

I. Introduction

RIVER and reservoir monitoring with in-situ sensor nodes has to work where field visits are expensive and power is scarce [1], [2]. A typical node samples temperature (T), specific conductance (EC), dissolved oxygen (DO) and pH every 5–30 minutes and sends the readings over a cellular or low-power wide-area network (LPWAN). Such nodes lose data. The InWaterSense deployment on the Sitnica river in Kosovo delivered 19,090 of its expected readings and missed 16,230 (46.0%) [3]; the CSIRO water-quality network in Australia averaged about 64% delivery because the gateway's connection to the server "was down occasionally" [4]; and deployment reports describe gateways that lose power every night [5], sinks that stop reconnecting until someone reboots them [6], [7], outages lasting days to weeks [8], and prolonged outages of cellular backhaul in remote areas [9].

There are three standard responses to a lost reading. A node can *retransmit* it until it is acknowledged (automatic repeat request, ARQ); it can *buffer* it and *backfill* it later; or the server can *reconstruct* it from the node's own history or from neighbouring sites. Which response is right depends on how data are actually lost. Retransmission helps against a short radio fade but wastes energy against a gateway that is down for a day. Reconstruction can fill a ten-minute gap but not a week. Yet most work on these remedies assumes a loss process that is short and memoryless. Estimation over lossy links is analysed with independent (Bernoulli) packet drops [10]–[12], and state-of-the-art imputation methods are evaluated on random masks or on simulated failures of a few hours [13]–[16]. Kong et al. [13] describe long losses caused by damaged or depleted nodes, but as named patterns, not as measured durations. How long real outages last, how much of the lost data they hold, and what that implies for the choice of remedy have not been measured for environmental telemetry.

This paper measures these quantities and draws the consequences, guided by four research questions:

- RQ1 How do real deployments lose data: how long are outages, how much of the lost data do long outages hold, and is a memoryless model adequate?
- RQ2 Under real outages, what do retransmission, buffering with back-off, and batching each recover, and at what energy and delay on LoRaWAN and NB-IoT?
- RQ3 Can reconstruction, from a site's own history or from neighbouring sites on the same river, replace delivery during an outage?
- RQ4 Can one reporting scheme deliver every reading, raise alarms as fast as per-reading reporting, and keep the server's real-time picture within a stated tolerance as often as per-reading reporting, while using substantially less energy than per-reading retransmission?

The contributions of this paper are as follows.

- *Measured outage processes* (Sec. IV). Records from four real systems, two water-quality deployments and two LoRaWAN networks, show that losses form two regimes: short radio losses, and rare system outages that hold most of the lost data. The memoryless (geometric, Gilbert) model fits worst on every record with an outage history, and no single distribution fits all of them.
- *A trace-driven comparison of loss handling* (Sec. V). Replaying the real delivery records through an energy model sourced from published LoRaWAN and NB-IoT measurements shows that ARQ costs 2.59×–7.35× the energy of unconfirmed reporting on LoRaWAN and recovers none of the outage losses; that buffering with probe back-

K. S. Adu-Manu and S. K. Azongo are with the Department of Computer Science, University of Ghana, Legon, Ghana. Corresponding author: K. S. Adu-Manu.

off delivers every reading at an energy within -14.6% to $+25.6\%$ of not recovering at all on LoRaWAN; and that hourly batching is $1.33\times$–$3.33\times$ cheaper with little loss of timeliness, because the outages, not the reporting scheme, cap timeliness.

- *The limits of reconstruction* (Sec. VI). Own-history and neighbour-based estimators are compared on 9 river chains (27 USGS sites). For DO and pH a site's own day–night (diel) pattern is the better real-time estimator; neighbours help temperature, and help most when bridging a gap after the data return. No method keeps day-long gaps within tolerance. A naive test against the last value is also shown to overstate the value of neighbours.
- *A reporting design, evaluated against pre-specified hypotheses* (Sec. VII). The design combines hourly batching, immediate uploads on a node-side event alarm, a bounded-error trigger that mirrors the server's estimate, and buffering with back-off. It is evaluated on 30 USGS river sites, three real outage records and three radio settings against hypotheses and decision rules written down before the runs, and every verdict is reported, including the failures.

The rest of the paper is organised as follows. Sec. II reviews related work, and Sec. III describes the data and models. Secs. IV–VII answer RQ1–RQ4 in turn, and Sec. VIII discusses practical guidance, validity and limitations.

II. Related Work

A. How sensor networks lose data

Early measurements of dense wireless sensor networks showed that packet delivery varies widely between links and over time [17]. Kong et al. [13] analysed three real sensor datasets with losses of 23%, 35% and 64%, and synthesised five named loss patterns, from random element loss to "successive loss in a row" when a node is damaged or out of energy; the patterns are motivated by their causes, but their durations are not measured. Deployment reports describe the long-outage regime anecdotally. In the CSIRO water-quality network, losses occurred at the gateway when its server connection was down, and end-to-end recovery from RAM copies improved delivery by at most 10.6% [4]. A solar-powered gateway lost power every night until after sunrise [5]; a GPRS sink failed to reconnect after disconnections until it was rebooted remotely [6]; a gateway's modem silently disappeared from its operating system until a daily watchdog reset was added [7]; and a sub-Arctic LoRaWAN deployment saw outages of days to weeks from gateway, power and connectivity faults [8]. For LoRaWAN, public datasets now allow loss to be measured from frame counters: LoED records the traffic of nine gateways in London [18], and a 16-month deployment at the University of Virginia (UVA) records ten sensors, three gateways and co-located weather [19]. No previous study appears to have measured outage durations across several such records, fitted models to them, and asked what they imply for the choice of loss-handling remedy.

B. Estimation and retransmission under loss

Kalman filtering with intermittent observations [10] established that estimation error stays bounded only if observations arrive often enough, under independent (Bernoulli) arrivals. Work on real-time remote estimation asks whether a failed packet should be retransmitted or replaced by a fresh measurement [11], [12], and finds switching-type optimal policies for linear systems with independent packet drops. Age of information formalises timeliness [20], and semantic-aware remote estimation schedules redundant sensors that observe the same source [21]. These works supply the vocabulary of the question studied here, retransmit or refresh, but not its setting: multi-day outages, battery-limited LPWAN radios, and environmental signals with strong daily cycles.

C. Reconstruction of missing sensor data

Compressive and low-rank data gathering exploit spatial and temporal correlation to recover readings that were not collected or were lost [13], [22]–[24]. Graph neural networks are the current state of the art for spatio-temporal imputation: GRIN [14] and SPIN [15] were evaluated mainly on random point losses and on simulated block failures of 12–48 steps (1–4 h at 5-minute steps), and deep compressed sensing for urban environmental monitoring on completely random losses [16]. The simulated failures thus last hours, not days, and the GRIN and SPIN evaluations include no per-site diel (seasonal-naive) baseline, although several of their signals have strong day–night cycles. River networks have a specific spatial structure, along the flow, that geostatistical models represent explicitly [25]. This paper tests the simplest version of this idea, upstream and downstream neighbours on the same river, against a site's own diel pattern.

D. Store-and-forward in practice, and LPWAN energy

Buffering on the node and forwarding later is established practice. The CSIRO team proposed flash buffering "to handle network outage better" [4]; the potato-field deployment planned node-side logging with a daily network-wide acknowledgement [5]; SeNDT stored all lake water-quality data on the node for later collection from the shore [26]; Arctic tundra nodes keep append-only logs and advance their upload position only after an acknowledgement [27]; sub-Arctic nodes oversample and buffer in flash [8]; and a disruption-tolerant gateway stores LoRaWAN messages while its cellular backhaul is down [9]. The same reports document the pitfalls: draining a backlog can starve live traffic [6], and the buffer's storage can itself fail [7], [27]. What is missing is a quantitative comparison of these remedies, on energy, completeness and timeliness, under measured outage processes. The energy model of this paper rests on published measurements: state-by-state LoRaWAN current profiles [28], NB-IoT energy per report with release assistance [29], and NB-IoT power states [30]. Analytical models describe the NB-IoT latency–energy trade-off [31], and side-by-side measurements compare LoRaWAN and NB-IoT energy in a developing-country setting [32].

E. Prediction-based reporting

Dual prediction schemes run the same predictor at the node and the server and transmit only when the server's prediction would leave a tolerance [33], [34]. They are usually evaluated with reliable or independently lossy links, and their purpose is to suppress reports. This paper uses the same mechanism for a different purpose: not to suppress reports, but to decide when an hourly batch must be sent early because the server's estimate would otherwise leave tolerance.

III. Data and Models

A. Delivery records

The study uses four real records of what a network delivered (Table I).

InWaterSense (Kosovo). Node 1 of the InWaterSense deployment on the Sitnica river [3], [35] reported about every 611 s from May 2015 to January 2016. The node sends each parameter as a separate message, seconds apart; the messages of one sampling instant are grouped into a sampling cycle, and a slot is treated as delivered if a cycle arrived for it. The node delivered 19,090 cycles and missed 16,230 slots (46.0%).

Weija (Ghana). The Weija deployment [2] logged 9 parameter streams from October 2018 to March 2019. Only records not marked as imputed count as delivered. Models are fitted to the conductivity stream (32.5-minute cadence), and the other streams are used to check that its pattern is typical. Even among the records not marked as imputed, 10.4% of the conductivity values carry more than the three decimals the logger otherwise records (up to 14), a sign of unflagged interpolation. Such values hide some short outages, so counts of short outages are lower bounds; long outages are not affected.

UVA (Virginia, USA). Ten LoRaWAN sensors reporting every 5 min through three gateways on The Things Network, from February 2024 to June 2025 [19], [36]. Missing frames are found from frame-counter gaps. A missing frame is classified as a *system* loss if no sensor at all reached the database in its 10-minute bin (network server, internet or the operators' logging pipeline), as a *gateway* loss if all of the sensor's serving gateways were silent, and as a *radio* loss otherwise.

LoED (London, UK). The LoRaWAN at the Edge Dataset [18], [37] logs 5,118,817 uplink frames from 8,503 device addresses heard by 9 gateways. The devices belong to other operators, so a frame counts as lost when no LoED gateway heard it, and most devices are simply far away. Only *well-heard* devices are therefore used, those with at least 200 frames and a radio loss below 20% (179 of 1,780), as a proxy for a node's own link, and classify a missing frame as gateway loss if none of the device's serving gateways logged anything in that hour.

B. Outage processes

A delivery record is reduced to alternating *up-runs* (consecutive delivered slots) and *outages* (consecutive missing slots), in units of the record's slot. Let $k_1, \ldots, k_n$ be the outage lengths and $h(k)$ the duration of an outage of k slots. Because the quantity that matters for loss handling is where the lost data are, not where the outage events are, the main statistic is the share of lost slots that fall in outages of at least a duration L,

$$S(L) = \frac{\sum_{i:\, h(k_i) \geq L} k_i}{\sum_{i=1}^{n} k_i}, \tag{1}$$

reported for $L = 6$ h and 1 day with bootstrap 95% confidence intervals [38] (outages resampled with replacement, 5,000 draws).

TABLE I
Delivery records. Loss: share of expected readings (InWaterSense, Weija) or frames (UVA, LoED) that did not arrive. Outages: runs of consecutive missing slots or frames. Lost in $\geq$1 d: share of the lost readings that fall in outages of one day or more, $S(1\text{ d})$ in (1). Weija: conductivity stream (71.0%–100.0% over all 9 streams). LoED: well-heard devices only (radio loss). GW: gateways.

Record	Type	Slot	Loss	Outages	Lost in $\geq$1 d
InWaterSense	water quality	611 s	46.0%	1,619	50.7%
Weija	water quality	32.5 min	15.4%	42	82.9%
UVA	LoRaWAN, 3 GW	5 min	4.5%	36,519	23.6%
LoED	LoRaWAN, 9 GW	varies	9.6%	56,276	–

Four models are fitted to the outage lengths $k \geq 1$ by maximum likelihood:

$$P_{\mathrm{geo}}(k) = (1-q)\, q^{k-1}, \tag{2}$$

$$P_{\mathrm{mix}}(k) = w\,(1-q_1)\, q_1^{k-1} + (1-w)\,(1-q_2)\, q_2^{k-1}, \tag{3}$$

$$P_{\ln}(k) = \frac{\Phi\!\left(\frac{\ln(k+\frac{1}{2})-\mu}{\sigma}\right) - \Phi\!\left(\frac{\ln(k-\frac{1}{2})-\mu}{\sigma}\right)}{1 - \Phi\!\left(\frac{\ln\frac{1}{2}-\mu}{\sigma}\right)}, \tag{4}$$

$$P_{\mathrm{pl}}(k) = \frac{k^{-\alpha}}{\zeta(\alpha)}, \tag{5}$$

where Φ is the standard normal distribution function and ζ the Riemann zeta function. The geometric model (2) is the outage-length distribution of the memoryless Gilbert two-state model; the two-geometric mixture (3) is equivalent to a Markov chain with one up state and two down states ("short loss" and "long outage"); (4) is a lognormal discretised to integer lengths; and (5) is a discrete power law. Models are compared by Akaike's information criterion, $\mathrm{AIC} = 2p - 2\ln\hat{\mathcal{L}}$ for p parameters and maximised likelihood $\hat{\mathcal{L}}$ [39], lower being better. Dependence between consecutive outages and up-runs is measured by Spearman's rank correlation.

C. Energy model and loss-handling strategies

The node is a Libelium Waspmote-class host [40] with one of three radios: NB-IoT (the N211 module measured in [29], with release assistance, RAI), LoRaWAN at spreading factor 7 (SF7), or LoRaWAN at SF12 in the EU868 band. Time is slotted at the reporting cadence and the node produces one reading per slot. Each transmission attempt a is charged the measured energy e_a of its radio and outcome:

NB-IoT. An unconfirmed report costs 0.31 J and a confirmed one 0.33 J (release after the uplink and after the reply, respectively [29]). A bulk frame of up to 512 B costs up to 60% more than a 20-byte report [29]. No published measurement was

found of the energy that an NB-IoT modem spends searching for a cell when there is no network; it is therefore modelled as the receive power of 0.222 W [30] times a search time T_{search}, and sweep $T_{\mathrm{search}} \in \{2.2, 10, 60\}$ s (default 10 s, 2.22 J per failed attempt).

LoRaWAN. Energies come from the state-by-state current profile of [28], with airtime computed from the payload. At SF7 the acknowledgement arrives in the first receive window, so a confirmed success (0.164 J for a 20-byte report) is cheaper than an unconfirmed report (0.244 J), which keeps both windows open; at SF12 the acknowledgement arrives in the second window and a confirmed success is dearer (0.858 vs 0.729 J). A failed confirmed attempt adds the acknowledgement time-out. Retransmissions lower the data rate every two attempts, as in the LoRaWAN specification [28]. Frames carry at most the regional maximum payload [41], and the 1% duty cycle delays successive frames; it costs time, not energy.

A reading in a bulk frame takes 12 B (four parameters, a time offset and a sequence number) and a frame 4 B of overhead, so a frame holds 42 readings on NB-IoT, 18 at SF7 and 3 at SF12. The host is awake for every attempt, at $P_{\mathrm{on}} = 69$ mW [40], and its awake time is added to e_a. Over a record of D days with sampling interval T_{s} (s), the transmission energy per day and the modelled lifetime on a battery of energy B are

$$E_{\mathrm{tx}} = \frac{1}{D}\sum_a e_a, \qquad Y = \frac{B}{365\,(E_{\mathrm{floor}} + E_{\mathrm{tx}})}, \tag{6}$$

$$E_{\mathrm{floor}} = 86{,}400\,(P^{\mathrm{radio}}_{\mathrm{sleep}} + P^{\mathrm{host}}_{\mathrm{sleep}}) + \frac{86{,}400}{T_{\mathrm{s}}}\,P_{\mathrm{on}}\,T_{\mathrm{sense}}, \tag{7}$$

where the floor (7) covers sleep and sensing: radio sleep power 9.35 μW for NB-IoT in power-saving mode [29] and 148 μW for the LoRa radio [28], host deep sleep 0.12 mW [40], and an assumed $T_{\mathrm{sense}} = 5$ s awake per sample for probe settling and reading. A 6,600 mAh battery at 3.7 V ($B = 87.9$ kJ) is assumed. Lifetimes are indicative; ratios between strategies are the robust result.

The following strategies are compared:

- *Unconfirmed*: one unconfirmed attempt per reading; a lost reading is never recovered.
- *Unconfirmed + back-off* (NB-IoT only): as above, but when the modem reports that there is no network the node backs off its attempts as in (8) and discards the readings it cannot send. This is the fair NB-IoT baseline: a node that searches for a cell in every slot of a day-long outage wastes energy for no benefit.
- *ARQ*: confirmed, up to 8 attempts per reading on LoRaWAN (the specification's default) or 3 on NB-IoT; 3 attempts on LoRaWAN as a sensitivity.
- *Confirmed + buffer + back-off* ("ARQ + buffer"): a reading that fails is buffered and the node enters outage mode, in which it sends one small confirmed probe at a time. After the n-th failed probe it waits

$$w_n = \min\,(2^n,\, K)\ \text{slots}, \qquad K = 6\ \text{h}, \tag{8}$$

  before the next, buffering the readings of the skipped slots without radio energy. When a probe succeeds, the buffer is uploaded in confirmed bulk frames.
- *Batching*: the node always buffers and uploads its buffer every 1 h or 6 h in confirmed bulk frames, with the back-off (8) when an upload fails.

Delivery records are replayed as the link state: a slot is *up* if the record delivered in it. Independent radio losses can be added per attempt, with probability $p_{\mathrm{radio}} = 0.096$ from LoED (Sec. IV-C). The reported metrics are the energy (6), the share of readings delivered, and the share of all readings that reached the server within 1 h.

D. The server's real-time picture and the tolerances

Between deliveries the server must show something. Let $x(t)$ be the true reading of one parameter at slot t, $A(\tau)$ the set of slots whose readings have arrived at the server by time τ, and $s^*(\tau) = \max A(\tau)$ the latest of them. The server shows

$$\hat{x}(\tau) = \begin{cases} x(\tau), & \tau \in A(\tau), \\ x(s^*) + \Delta(\tau), & \text{otherwise}, \end{cases} \tag{9}$$

where the diel correction $\Delta(\tau) = x(\tau - 24\,\mathrm{h}) - x(s^* - 24\,\mathrm{h})$ if both of these readings are in $A(\tau)$, and $\Delta(\tau) = 0$ otherwise, that is, the latest arrived reading plus the site's own change over the same hours one day earlier. This *own-diel* estimator is the best simple real-time estimator in the river data used here (Sec. VI). The picture is scored as the share of slots at which $|\hat{x}(\tau) - x(\tau)| \le \delta$, for a tolerance δ per parameter. Three tolerance levels are used: at the reference level L2, δ is 0.2 °C for T, 2% saturation for DO, 0.1 for pH and 2% of the record's median for EC; the tight level L1 halves these (0.1 °C, 1%sat, 0.05, 1%), and the loose level L3 widens them (0.5 °C, 5%sat, 0.2, 5%).

E. River water-quality data

River signals come from the U.S. Geological Survey's National Water Information System [42]: 2023 instantaneous values of T, EC, DO and pH, approved values only. DO concentrations are converted to percent saturation with the Benson–Krause equation [43] at the measured water temperature. Sec. VI uses 9 river chains, sets of gauging sites on the same river (27 sites); Sec. VII uses every downloaded site that has all four parameters in complete cycles for at least half of the year (30 of 39 sites).

IV. How Real Deployments Lose Data

A. Water-quality deployments: most lost data are in long outages

Fig. 1 shows when each record was down. Two features are visible at once: many short gaps, and a few gaps lasting days.

InWaterSense node 1 had 1,619 outages. Many were short: 47.3% lasted a single slot, but together they held only 4.7% of the lost readings. At the other end, 14 outages of a day or more (0.9% of all outages) held 50.7% of the lost readings (bootstrap 95% CI 29.4%–64.6%), and outages of 6 h or more held 68.5%

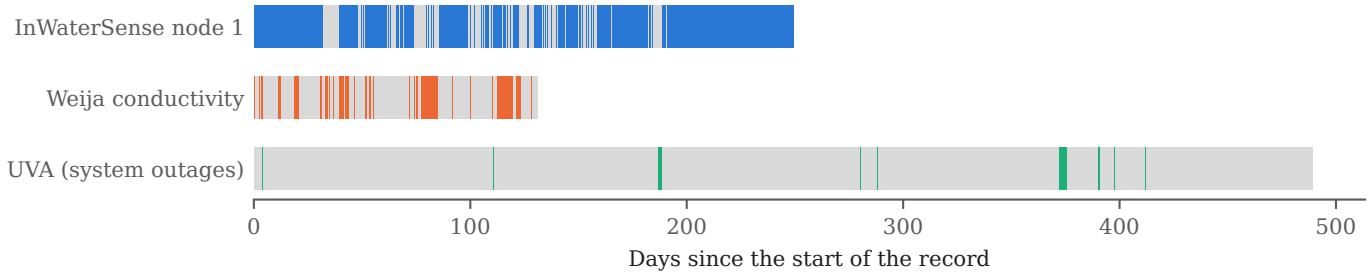


Fig. 1. Delivery timelines of the three records with outage histories: coloured spans are slots in which nothing was delivered (UVA: system outages only; its short radio losses are not shown). Each record is drawn on its own time axis, from its first slot.

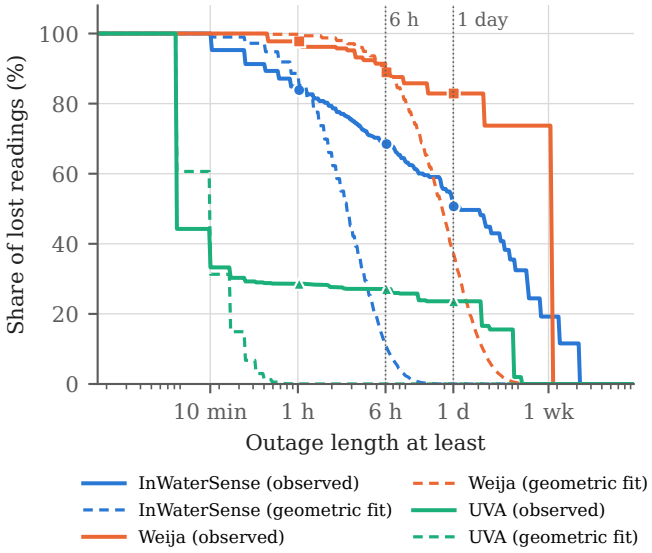


Fig. 2. Share of lost readings that fall in outages at least as long as the value on the horizontal axis, $S(L)$ in (1) (solid), and the same share predicted by the geometric (memoryless) model (2) fitted to each record's outage lengths (dashed). Markers at 1 h, 6 h and 1 day. InWaterSense node 1 (611-s slots), Weija conductivity (32.5-min slots) and UVA (all sensors, 5-min slots; radio and system losses together).

(54.9%–77.2%). The longest lasted 13.3 days. Fig. 2 shows the whole curve.

The Weija deployment, on a different continent, shows the same structure. Its conductivity stream had 42 outages and lost 15.4% of its readings; 82.9% of the lost readings fell in outages of a day or more, and the share was between 71.0% and 100.0% across all 9 streams, the longest outage lasting 36.7 days. With only 3 day-long outages in the conductivity stream, its confidence interval is wide (0.0%–93.5%), so Weija is treated as qualitative support for the InWaterSense result rather than as a second estimate of it.

B. The memoryless model is wrong, and no single model is right

Table II compares the four outage-length models (2)–(5). The geometric model (2), i.e., the memoryless loss assumed by independent-loss and Gilbert models, fits worst on every

TABLE II
AIC OF OUTAGE-LENGTH MODELS (LOWER IS BETTER; BEST PER RECORD IN BOLD). UVA: ALL MISSING-FRAME RUNS OF ALL SENSORS, 5-MIN SLOTS.

Model	InWaterSense	Weija	UVA
Geometric (memoryless)	10,537	341	76,881
Two-geometric mixture	7,107	236	**32,966**
Discrete lognormal	**6,859**	222	33,327
Discrete power law	6,863	**221**	33,180

record, by 3,677 AIC points on InWaterSense, 120 on Weija and 43,915 on UVA. Its failure is in exactly the quantity that matters, the long-outage share (1): fitted to InWaterSense, it puts 11.4% of the lost readings in outages of 6 h or more, against 68.5% observed, and less than 0.001% in outages of a day or more, against 50.7% (Fig. 2). On Weija it puts 36.9% in day-long outages, against 82.9%.

No single alternative wins everywhere. The discrete lognormal is best on InWaterSense, the discrete power law on Weija (with the lognormal 1.7 AIC points behind), and the two-geometric mixture on UVA. The mixture's form, many short losses and a few long outages, is the common thread, and the records differ visibly in how many long outages they have (Fig. 2). Outages are also not independent of each other. On InWaterSense a long outage tends to be followed by another long outage (Spearman $\rho = +0.27$) and by a short up-run ($\rho = -0.20$; both $p < 0.001$), so outages cluster into bad periods that a renewal model, drawing each outage independently, cannot reproduce. On Weija, with 42 outages, neither correlation is distinguishable from zero ($\rho = +0.02$, $p = 0.88$; $\rho = -0.06$, $p = 0.73$). For these reasons, all simulations in this paper replay the real records rather than a fitted model.

C. LoRaWAN networks: short radio losses and rare system outages

The two LoRaWAN records separate the causes of loss. On LoED, well-heard devices lost 9.6% of their frames to the radio. Losses were mildly bursty: the probability of losing a frame after a lost frame was 0.25, about 2.6 times the marginal rate. But the bursts were short: of 56,276 radio-loss bursts, 84% were a single frame and 99% at most five frames.

Losses also did not coincide across devices beyond what their density implies: a lost frame coincided with a loss of another device at a shared gateway within the same minute in 19.2% of cases, against 18.2% (16.7%–19.5%) under a random time-shift baseline.

UVA adds what LoED, recorded in campaign windows, cannot show: 16 months of continuous operation. The system as a whole was available 98.72% of the time, with 12 system outages (median 1.67 h, longest 82.2 h). Of the 58,211 missing frames, 71.6% were radio losses and 28.4% system losses. The radio regime looks like LoED's: 89% of the 36,519 runs of missing frames were a single frame. The system regime looks like the water-quality deployments': 83% of the system down-time fell in outages of a day or more. With few system outages, UVA's long outages hold a smaller share of its lost frames than at InWaterSense (27.1% in runs of 6 h or more, 23.6% in runs of a day or more; Fig. 2).

Two further observations matter for design. First, *gateway diversity masks gateway outages*: one of UVA's gateways was down for 29.9 days in a row, yet none of the missing frames was a gateway loss, because every sensor was also heard by a second gateway. Second, rain had no detectable effect on link loss: over 18 sensor–gateway pairs, loss was 9.18% in dry hours and 9.21% in rainy hours (Wilcoxon signed-rank test [44], $p = 0.64$).

In answer to RQ1, real deployments lose data in two regimes. Radio losses are short and nearly independent; system outages, of a gateway, its backhaul or the server, are rare but long. In the two water-quality deployments, outages of 6 h or more hold most of the lost data (68.5% on InWaterSense, 88.9% on Weija conductivity). The memoryless model misses the second regime entirely.

V. Retransmit or Backfill?

The InWaterSense and Weija conductivity records are replayed as the link state through the energy model of Sec. III-C, for NB-IoT (search times T_{search} of 2.2, 10, 60 s), LoRaWAN SF7 and SF12, without and with independent radio losses ($p_{\mathrm{radio}} = 0$ and 0.096): 20 configurations in all. Every slot in which a record delivered nothing is a slot in which the link is down. These records cannot separate radio losses from outages, so this replay is pessimistic for ARQ, whose retries in such a slot all fail; the independent radio losses added on top are the part that ARQ can recover. Table III shows the InWaterSense record; ranges in the text are over all applicable configurations. (Mapped onto the simulator's regular slot grid, the InWaterSense record loses 46.1% of the readings, against 46.0% counted directly.)

A. ARQ pays for losses it cannot recover

Without added radio loss, ARQ delivers exactly what unconfirmed reporting delivers: every reading lost to an outage is lost after all of its retries. On LoRaWAN it costs 2.59×–7.35× the energy of unconfirmed reporting with 8 attempts, and 1.51×–2.94× with 3. On InWaterSense at SF7 this is 447.1 J/day instead of 61.6 J/day, a modelled lifetime of 0.46 instead of 1.80 years. With independent radio losses added, ARQ recovers them, raising the delivered share by $+5.0$ to $+7.9$ percentage points, but none of the outage losses. Retransmission is the right tool for the radio regime and the wrong one for the outage regime, which holds most of the lost data.

B. On NB-IoT, back-off is where the energy is won

An NB-IoT node that tries to send in every slot of an outage searches for a cell each time. Searching in every slot costs 1.20×–6.53× the energy of backing off (unconfirmed reporting without back-off against unconfirmed reporting with it), the ratio growing with the share of time in outages and with T_{search}. Because no published measurement of T_{search} was found, this is a sweep, not an estimate; with back-off, however, a node in a long outage searches at most once every 6 h instead of once per slot. The fair NB-IoT baseline is therefore unconfirmed reporting *with* back-off, not without it.

C. Buffering buys completeness at a modest energy cost

Confirmed reporting with a buffer and back-off delivers every reading in every configuration. Relative to the fair baseline that recovers nothing, its energy changes by -5.5% to $+15.3\%$ on NB-IoT and by -14.6% to $+25.6\%$ on LoRaWAN. The sign depends on the radio. At SF7 buffering ranges from -14.6% to $+0.6\%$, because a confirmed success, with its acknowledgement in the first receive window, costs less than an unconfirmed report (Sec. III-C), and because the node stops transmitting into outages. At SF12 it is always dearer, by $+7.8\%$ to $+25.6\%$, because the acknowledgement arrives in the second window. On InWaterSense at SF7 it uses 52.7 J/day (1.93 years) and delivers all readings, against 61.6 J/day and 1.80 years for unconfirmed reporting, which loses 46.1% of them (Table III).

D. Batching is cheaper, and costs little timeliness

Bulk frames carry many readings for little more than the cost of one (Sec. III-C). Hourly batching with buffer and back-off delivers every reading and uses 1.33×–3.33× less energy than the fair per-reading baseline, 18.9 J/day (2.65 years) on InWaterSense at SF7. Its share of readings that reach the server within 1 h differs from unconfirmed reporting's by only -3.3 to $+7.3$ percentage points, because under real outages the outages, not the batching interval, decide how many readings arrive within the hour. Six-hourly batching is cheaper still, but delivers only a small share of readings within the hour (Table III).

The price of batching is the timeliness of individual readings: a reading can wait up to an hour, and an hour can matter. It matters for alarms, when something happens in the river, and for the server's real-time picture of fast-changing parameters. Sec. VII addresses both.

In answer to RQ2, ARQ is the wrong remedy under real outages: it costs several times the energy and recovers only the short radio losses. Buffering with back-off recovers everything at an energy within about a quarter of that of recovering nothing (-14.6% to $+25.6\%$ on LoRaWAN, -5.5% to $+15.3\%$

TABLE III
LOSS-HANDLING STRATEGIES ON THE INWATERSENSE RECORD (NO ADDED RADIO LOSS): TRANSMISSION ENERGY PER DAY, E_{tx} IN (6), FOR EACH RADIO (NB-IOT: N211 WITH RAI, $T_{\mathrm{search}} = 10$ S), AND THE SHARE OF READINGS DELIVERED AND DELIVERED WITHIN 1 H (LORAWAN SF7; NB-IOT FOR THE NB-IOT-ONLY STRATEGY). BACK-OFF AS IN (8), CAPPED AT 6 H. –: NOT APPLICABLE.

	Energy (J/day)			Readings	
Strategy	NB-IoT	SF7	SF12	delivered	within 1 h
Unconfirmed, every reading	229.9	61.6	147.2	53.9%	53.9%
Unconfirmed + back-off (NB-IoT only)	59.5	–	–	47.1%	47.1%
ARQ (8 attempts LoRaWAN; 3 NB-IoT)	231.5	447.1	796.1	53.9%	53.9%
ARQ, 3 attempts (LoRaWAN)	–	175.5	354.3	53.9%	53.9%
Confirmed + buffer + back-off	63.8	52.7	158.7	100.0%	52.8%
Hourly batch + buffer + back-off	23.6	18.9	89.5	100.0%	52.0%
6-hourly batch + buffer + back-off	13.3	11.3	79.0	100.0%	11.2%

on NB-IoT), and hourly batching is the energy-efficient way to do it, at a timeliness cost that only alarms and fast parameters notice.

VI. CAN RECONSTRUCTION REPLACE DELIVERY?

If a server could estimate the missing readings well enough, it would not need them delivered. This is tested on 9 river chains: sets of USGS gauging sites along the same river (Delaware, Illinois, Tualatin, Clackamas, McKenzie, Monongahela, Youghiogheny, Schuylkill and Klamath; 27 sites in all), each pair of sites on a chain serving as target and neighbour in both orders (53–57 ordered pairs per parameter).

A. Method

All series are placed on a common 1-h grid. For every ordered pair, parameter and gap length (1, 3, 6, 12, 24, 48, 72 h), gaps are simulated at the target (a median of 300 per pair, parameter and length), drawn from the last 80% of the year. A gap covers the slots after the last observed slot t_0 and before the next observed slot t_1. Let y be the neighbour's series, lagged by ℓ hours, and b a scaling slope; $\ell \in [0, 48]$ h is chosen to maximise the correlation between 6-h changes of target and lagged neighbour in the first 20% of the year, and b is the least-squares slope of those changes (pairs whose best correlation is negative are dropped). *Real-time* estimators use only data up to t_0, and the neighbour during the gap:

$$\hat{x}_{\mathrm{pers}}(t) = x(t_0), \tag{10}$$

$$\hat{x}_{\mathrm{diel}}(t) = x(t_0) + x(t - 24\,\mathrm{h}) - x(t_0 - 24\,\mathrm{h}), \tag{11}$$

$$\hat{x}_{\mathrm{nb}}(t) = x(t_0) + b\,[y(t) - y(t_0)], \tag{12}$$

$$\hat{x}_{\mathrm{dn}}(t) = \hat{x}_{\mathrm{diel}}(t) + b\,[y(t) - y(t_0)] - b\,[y(t - 24\,\mathrm{h}) - y(t_0 - 24\,\mathrm{h})]. \tag{13}$$

Persistence (10) holds the last value; own diel (11) is the server's estimator (9); neighbour change (12) adds the neighbour's scaled change since the gap began; and diel + neighbour (13) adds only the part of that change which the neighbour's own diel pattern does not explain. *Bridging* estimators also know $x(t_1)$, as they would once backfilled data arrive:

$$\hat{x}_{\mathrm{int}}(t) = x(t_0) + \frac{t - t_0}{t_1 - t_0}\,[x(t_1) - x(t_0)], \tag{14}$$

$$\hat{x}_{\mathrm{in}}(t) = \hat{x}_{\mathrm{int}}(t) + b\,[y(t) - \bar{y}(t)], \tag{15}$$

where $\bar{y}$ is the straight line from $y(t_0)$ to $y(t_1)$, so that (15) adds the neighbour's departure from its own line to the interpolation (14). The error of an estimator on a gap is its maximum absolute error over the gap, $\max_{t_0<t<t_1} |\hat{x}(t) - x(t)|$; the median over gaps is reported, together with the share of gaps whose maximum error is within the L2 tolerance.

B. Results

Table IV gives the results for 1-h and day-long gaps. The text reports 24-h gaps; relative gains are medians over pairs, and exclude the 3 pH pairs whose target (site 14206694) has a median error of exactly zero, for which a relative gain is undefined.

In real time, a site's own diel pattern is the estimator to beat. It beats persistence in 77% of pairs for DO and 84% for T. A neighbour improves on the better of the two own-data estimators in only 30% of pairs for DO (median gain -8.8%), 9% for pH (-6.6%) and 23% for EC (-1.9%). Temperature is the exception: a neighbour helps in 72% of pairs (median gain $+6.5\%$), consistent with a shared, non-periodic weather driver.

When bridging a gap after the data return, the neighbour helps more. The neighbour's shape improves interpolation in 93% of pairs for T (median gain $+23.4\%$) and 70% for DO ($+10.3\%$), but not for pH (38% of pairs, -0.2%) or EC (25%, -2.2%).

No method keeps day-long gaps within tolerance. For 1-h gaps, the better own-data real-time estimator keeps a median of 96% of gaps within the L2 tolerance. For 24-h gaps, even the best of all six estimators, including the bridging ones that already know the value after the gap, keeps a median of only 26% of gaps within tolerance (upper quartile 61%).

A naive baseline overstates the value of neighbours. Judged against persistence alone, a neighbour method wins in 77% of pairs for DO and 36% for pH, and the best neighbour method cuts the median DO error from 9.07 to 5.13 %sat. Against the target's own diel pattern, which reaches 4.33 %sat, most of that gain disappears: the neighbour wins in 30% of DO pairs and 9% of pH pairs. Much of what a naive test credits to spatial correlation is the shared day–night cycle, which the target's own history already contains. Evaluations of spatio-temporal reconstruction on river data therefore need a per-site diel baseline.

TABLE IV
RECONSTRUCTION ON 9 RIVER CHAINS: MEDIAN OVER ORDERED SITE PAIRS OF THE MEDIAN MAXIMUM ABSOLUTE ERROR OVER A GAP (T IN °C, DO IN %SAT, PH IN PH UNITS, EC IN μS/CM), AND THE SHARE OF PAIRS IN WHICH THE NEIGHBOUR IMPROVES ON THE TARGET'S OWN DATA. REAL TIME: BEST NEIGHBOUR METHOD, (12) OR (13), VS THE BETTER OF PERSISTENCE (10) AND OWN DIEL (11). BRIDGING: INTERPOLATION + NEIGHBOUR (15) VS INTERPOLATION (14).

		Real time				Bridging		
Param.	Gap (h)	Persistence	Own diel	Best neighbour	Neighbour wins	Interpolation	Interp. + neighbour	Neighbour wins
T	1	0.07	0.05	0.05	23%	0.03	0.03	19%
T	24	0.97	0.70	0.65	72%	0.80	0.56	93%
DO	1	0.67	0.49	0.58	19%	0.26	0.34	13%
DO	24	9.07	4.33	5.13	30%	7.75	6.27	70%
pH	1	0.000	0.025	0.018	7%	0.013	0.013	9%
pH	24	0.175	0.125	0.149	9%	0.172	0.172	38%
EC	1	0.50	0.50	0.66	16%	0.25	0.27	16%
EC	24	7.50	8.25	7.50	23%	4.44	4.42	25%

In answer to RQ3, reconstruction cannot replace delivery. For DO and pH the best real-time estimate is the site's own diel pattern, and neighbours add little; for temperature, and for bridging gaps after the data return, neighbours help. For gaps of a day, which hold most of the lost data (Sec. IV), no estimator keeps the error within tolerance. Reconstruction's role is the interim estimate between deliveries, and only delivery restores the data. This is the role it plays in the design of Sec. VII.

VII. A REPORTING DESIGN AND ITS EVALUATION

Secs. IV–VI point to a design: buffer and back off through outages (only delivery restores long gaps), batch routine data (cheap, and little less timely under real outages), and show the site's own diel estimate between deliveries. Batching leaves two gaps, alarms and the real-time picture of fast parameters, and the design closes them with two triggers that send a batch early.

A. The design

Event-driven batching runs at the node (Algorithm 1, without the bounded-error trigger). Every reading is buffered. Once an hour the node uploads its buffer in confirmed bulk frames. When the node's event detector raises an alarm, it uploads the buffer at once, alarming reading included; if the node is in an outage, an alarm triggers an immediate probe outside the back-off schedule (without resetting it). A failed upload puts the node into outage mode, with the probe back-off (8).

Fig. 3 shows these behaviours in a short illustrative scenario run through the simulator. Before the outage, the alarm makes the event-driven node upload at once, where plain batching would wait for the end of the hour; the alarm during the outage cannot get through; and after the outage, when the backlog arrives depends on where each strategy is in its back-off (8).

Bounded event-driven batching adds a *bounded-error trigger*. Because every upload is confirmed, the node knows which of its readings have reached the server and when; it can therefore compute, at every slot t, exactly the value $\hat{x}_j(t)$ that the server is showing for each parameter j, by evaluating (9) on the readings it knows to have arrived. It uploads its buffer at once (Algorithm 1) if

$$\max_j \frac{|x_j(t) - \hat{x}_j(t)|}{\delta_j} > 1, \tag{16}$$

that is, if any parameter's true reading differs from what the server shows by more than its tolerance δ_j. This is the dual-prediction mechanism of error-bounded report suppression [33], [34], used here to bound the error between batches rather than to suppress reports. Two details matter. First, the trigger is not consulted during an outage (a choice fixed before the evaluation): the server's estimate may drift far from the truth during a long outage, and a node that probed every time would waste energy on a dead link; only alarms probe. Second, the node's copy of the server must model *when* readings arrive, not only *whether*: on LoRaWAN the duty cycle spaces the frames of a long upload, so a reading enters $A(t)$ in (9) only after its frame's arrival time. In the final design, the newest valid reading goes in the first frame of every upload, so that the value the server most needs arrives first; the runs for H1–H7 used first-in-first-out packing (Sec. VII-G).

Algorithm 1 Bounded event-driven batching: node, one slot

```
append the slot's reading x_t to the buffer
if in outage mode then
    if alarm at t or back-off timer expired then
        probe; on success leave outage mode and upload the buffer
    end if
    return
end if
x̂(t) ← server's estimate (9), from readings whose arrival time is ≤ t
if alarm at t or trigger (16) or hour boundary then
    upload the buffer (newest valid reading first), confirmed bulk frames
    on failure enter outage mode, back-off (8)
end if
```

B. Evaluation protocol

The evaluation followed a plan written before the runs, with hypotheses, metrics and decision rules fixed in advance; every

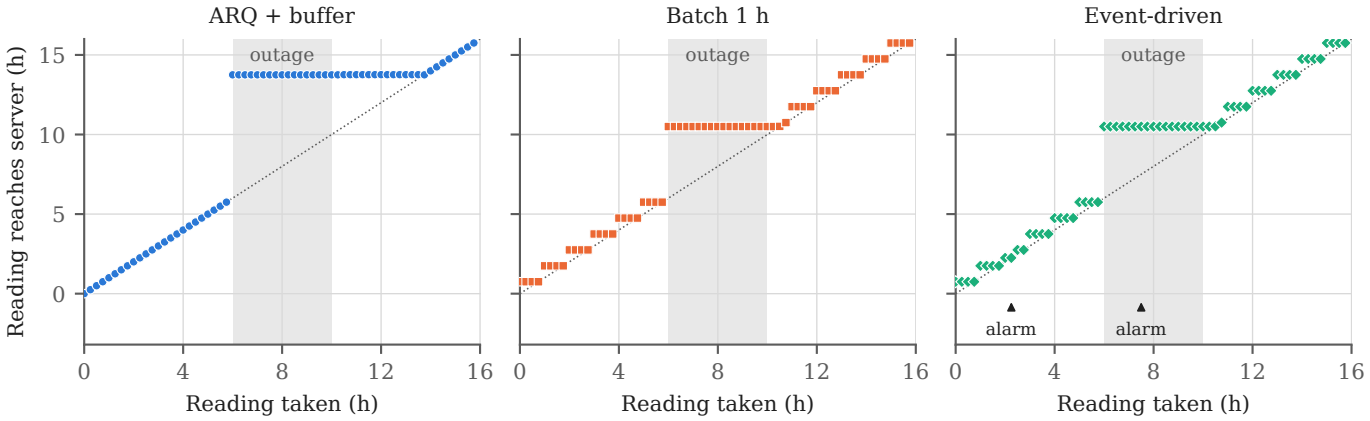


Fig. 3. When each reading reaches the server, in an illustrative scenario run through the simulator: LoRaWAN SF7, 15-min sampling, the link down from 6 to 10 h, and node alarms at 2.25 h (link up) and 7.5 h (link down). Points on the dotted diagonal arrive at once; horizontal runs are readings delivered together. Event-driven batching follows hourly batching except that the alarm at 2.25 h sends its batch early.

verdict is reported, together with every deviation from the plan and its reason.

Sites. All 30 USGS sites (of 39 downloaded) with all four parameters in complete cycles for at least half of 2023; the others were excluded for sparse data (7), no approved pH (1), or only 72 days of data (1; found after the first run and excluded before any verdict). Sampling steps are 5 min (1 site), 15 min (21), 30 min (7) and 60 min (1). The first 20% of each record trains the detector; all metrics are computed on the remaining 80%.

Links. The no-outage link, and three real records mapped onto each site's time grid in physical time, wrapped circularly with a random start offset per site (seeded by the site's identifier): InWaterSense and Weija conductivity, in which every missing slot is a down link, and UVA, replayed as its system outages plus independent radio losses at the rate of a representative sensor (3.08% per attempt), because UVA's radio losses, unlike the other records' losses, can be separated from its outages. *Radios:* NB-IoT ($T_{\text{search}} = 10$ s), LoRaWAN SF7 and SF12.

Strategies. Unconfirmed, ARQ + buffer, hourly batching ("batch 1 h"), event-driven batching, and bounded event-driven batching (L2 tolerance), as defined above and in Sec. III-C.

Injected events. Events are added to the real test-period series, so background dynamics, noise and gaps are real and only the event is synthetic. For parameter j, let $a_j(t)$ be the reading minus its centred 24-h running median, and $r_j(t)$ that anomaly minus its median for the same calendar month and hour of day. The event scale is the robust standard deviation of r_j in the event's month,

$$\sigma_j = \max\left(1.4826\ \mathrm{MAD}[r_j]\,,\ \rho_j\right), \tag{17}$$

floored at the recording resolution ρ_j, estimated from the data as the most common positive difference between sorted distinct values. An event of amplitude $\kappa\sigma_j$, with σ_j from (17), $\kappa \in \{5, 8\}$ and a random sign, lasts 180 min: a *step* adds $\kappa\sigma_j$ for its whole duration, and a *ramp* rises linearly from 0 to $\kappa\sigma_j$; injected values are rounded to ρ_j. Events touch one parameter each, in turn, lie in the test period, at least a day after the start of a data segment and wholly inside it, and at least three days apart (at most 60 per site and setting): 1,682 events in each of the four settings (6,728 in all).

Detector. All strategies use the same causal detector: on the node for the event-driven designs, and on the server, on the readings it has received, for the others. It compares each reading with the reading $m = 120$ min earlier, corrected by a learned hour-of-day profile $p(\cdot)$ of the departure from the running 24-h median, and runs a two-sided CUSUM [45] on the clipped, standardised residual:

$$z_t = \mathrm{clip}\left(\frac{x_t - x_{t-m} - p(h_t) + p(h_{t-m})}{s},\ -c,\ c\right), \tag{18}$$

$$S_t^+ = \max\left(0,\ S_{t-1}^+ + z_t - k\right),$$
$$S_t^- = \max\left(0,\ S_{t-1}^- - z_t - k\right), \tag{19}$$

where h_t is the hour of day and s a robust residual scale floored at ρ_j. An alarm is raised when either sum in (19), driven by the residual (18), exceeds H, with $k = 3$, $H = 8$ and $c = 6$; the detector then stays in an event state until $|z_t| < 2$ for 60 min, and resets its sums. The profile and the scale are initialised on the training period and updated by exponential smoothing (memory about 7 days) only outside the event state, so that events do not teach the detector what is normal. These parameters were fixed before this evaluation and were not tuned on its data. An event counts as detected if an alarm on its parameter occurs during it. The detector caught a median (over sites) of 63% of 5σ steps, 90% of 8σ steps, 14% of 5σ ramps and 65% of 8σ ramps (57.4% of all events), with a median of 0.8 false alarms per parameter per 30 days (mean 1.4). These rates are the detector's, the same for every strategy; the comparison concerns how fast the server learns of a detected event.

Metrics. Alarm latency: the time from an event's onset to the arrival of its alarm at the server, split by whether the link was up at onset. *Energy*: transmission J/day, E_{tx} in (6), including uploads caused by alarms and false alarms; for the event-driven designs, averaged over the four event settings. *Picture*: the share of scored slots at which the server's real-time picture

(9) is within tolerance, on the series without injected events. Each is summarised as the median over sites, per link and radio (12 link–radio combinations; 48 with the four parameters).

Table V lists the hypotheses and their verdicts. H1–H4 were fixed for event-driven batching. After H4 failed, H5–H7 were written for the bounded-error trigger before it was implemented, and H8–H10, on packing order and on the tolerance level, before they were run. The runs for H1–H7 packed uploads first-in-first-out; newest-first packing was evaluated separately (H8). Four deviations from the plan were recorded when made: UVA replayed as system outages plus radio losses (D1, after a one-site test); one site excluded because its eligibility had been computed over its record span rather than the year (D2); seeds derived from site identifiers rather than list positions, with a full rerun (D3); and two implementation defects of the packing order, fixed with a full rerun (D4, below). Every H1–H7 verdict was the same before and after the reruns.

C. Alarms with the link up (H1, H7)

With the link up at an event's onset, event-driven batching raises alarms at the server as fast as per-reading ARQ (H1 passes, 12/12; Table VI). The median latency of about an hour is the detector's own: the CUSUM needs about an hour of evidence, and delivery adds nothing. Plain hourly batching is slower in every combination, by a median of 0.50 h over the combinations (Wilcoxon signed-rank test across sites, largest $p = 3 \times 10^{-6}$). On the InWaterSense link the event-driven design is faster than ARQ + buffer on every radio, because an alarm probes at once while ARQ + buffer waits for its back-off schedule when an outage starts during the event. Of all injected events, the server knows 33.3% within an hour with event-driven batching on the no-outage link (SF7), the same as with per-reading reporting, against 20.7% with hourly batching; on InWaterSense the shares are 18.4%, 16.5% (ARQ + buffer), 11.7% (unconfirmed) and 11.3% (batch). The ceiling is the detector's 57.4% eventual detection, not the reporting scheme. The bounded-error trigger does not delay alarms (H7 passes, 12/12).

D. Energy (H2, H6)

Alarm uploads are rare (a median of 0.10 alarms per node per day; mean 0.18), and most ride on readings that would have been sent within the hour anyway. Event-driven batching therefore costs 1.000–1.020 times as much as plain batching, and ARQ + buffer costs 1.41×–3.86× as much as event-driven batching (H2 passes, 12/12). The bounded-error trigger costs more: 1.07×–1.55× the energy of plain batching, because it fires in the hours when DO and pH change fastest. It remains 1.30×–2.32× cheaper than ARQ + buffer (H6 passes, 12/12). At the one hourly-sampled site (01463500), an hourly batch holds a single reading, and the bounded design costs between 0.05% more and 5% less than per-reading ARQ: batching gains little at a cadence of one hour or slower.

E. The real-time picture (H4, H5)

Table VII and Fig. 4 show the server's picture. Plain event-driven batching fails H4 (29/48): for DO and pH, an estimate up to an hour old leaves the L2 tolerance between batches, and the picture is -10.8 to -5.1 percentage points less often within tolerance than with per-reading reporting. The failures are all DO and pH combinations on the no-outage, UVA and Weija links, and DO at SF12 on InWaterSense (-5.07 percentage points). They occur with no outage at all, so they are a property of hourly batching, not of the outages. The event detector does not help: these excursions are ordinary diel motion, not events.

The bounded-error trigger removes the failure. The bounded design's picture is within -0.5 to $+0.7$ percentage points of per-reading reporting's for every parameter, link and radio (H5 passes, 48/48). With the link up and no radio loss it is within tolerance by construction, apart from the duty-cycle delays addressed by packing order (H8 below).

Under the two water-quality outage records, unconfirmed reporting shows a *better* picture than ARQ + buffer for every parameter and radio (e.g., DO on InWaterSense at SF7: 78.5% vs 73.8%). When the link returns, an unconfirmed node delivers its next reading at once, while a buffering node waits for its next back-off probe. The back-off saves energy and costs timeliness right after outages; unconfirmed reporting, of course, never delivers the readings lost in the outage.

F. Events that start during an outage (H3)

Table VIII and Fig. 5 show the detected events whose onset fell in an outage. As expected (H3), every strategy has to wait for the link. Unconfirmed reporting learns of almost none of these events (at most 4.3% on any link and radio): the alarming readings fall inside the outage and are lost. The buffering strategies learn of at least 98.4%, but only after the outage: a median of 21–60 h after onset, depending on the strategy, link and radio. The outage, not the reporting scheme, sets this latency. Within it, the immediate alarm probe makes event-driven batching faster than ARQ + buffer on InWaterSense, by 2.3–4.4 h depending on the radio, and on Weija, where it ties with plain batching, by 3.8–4.0 h. On UVA, whose outages are short, ARQ + buffer is faster by 0.5 h. The bounded design gives up part of this advantage; its median latency is never more than 0.5 h behind ARQ + buffer's.

G. Packing order and tolerance level (H8–H10)

On SF12 a frame holds only 3 readings, so an hourly batch at 15-min sampling needs two frames, and the duty cycle delays the second. With first-in-first-out packing, the newest reading rode in the second frame, and the server's picture on the no-outage link reached only 98.72% at worst instead of 100%. H8 predicted that packing the newest reading first would make it exact everywhere. By its pre-specified rule, H8 failed: the picture fell short at one site (01467200, 5-min sampling, SF12), with a minimum of 99.998%. The cause was two implementation defects, each a departure from the stated design: the reading promoted to the first frame could

TABLE V
HYPOTHESES, DECISION RULES (FIXED BEFORE THE RUNS) AND VERDICTS. RULES APPLY TO THE MEDIAN SITE OF EVERY LINK–RADIO COMBINATION (12), OR OF EVERY LINK–RADIO–PARAMETER COMBINATION (48). "EVENT" = EVENT-DRIVEN BATCHING; "BOUNDED" = BOUNDED EVENT-DRIVEN BATCHING.

	Claim	Rule	Verdict
H1	Link up: Event raises alarms as fast as per-reading reporting	Event − ARQ+buffer alarm latency $\leq$ 1 slot	Pass, 12/12
H2	Event costs about as much as batching, much less than per-reading ARQ	Event / batch $\leq$ 1.10 and ARQ+buffer / Event $\geq$ 1.3	Pass, 12/12
H3	Onset in an outage: buffering delivers late, unconfirmed loses	descriptive	As expected
H4	Event's picture is as good as per-reading reporting's	Event $\geq$ ARQ+buffer − 5 pp	Fail, 29/48
H5	Bounded's picture is as good as per-reading reporting's	Bounded $\geq$ ARQ+buffer − 1 pp	Pass, 48/48
H6	Bounded costs no more than per-reading ARQ	Bounded $\leq$ ARQ+buffer energy	Pass, 12/12
H7	The trigger does not delay alarms	Bounded − Event alarm latency $\leq$ 1 slot	Pass, 12/12
H8	Newest-first packing makes the picture exact on every radio	100% within tolerance, no outage; energy within ±1%	Fail; pass after fix (post hoc)
H9	H5 holds at tolerances L1 and L3	as H5	Pass, 96/96
H10	H6 holds at tolerances L1 and L3	as H6	Pass, 24/24

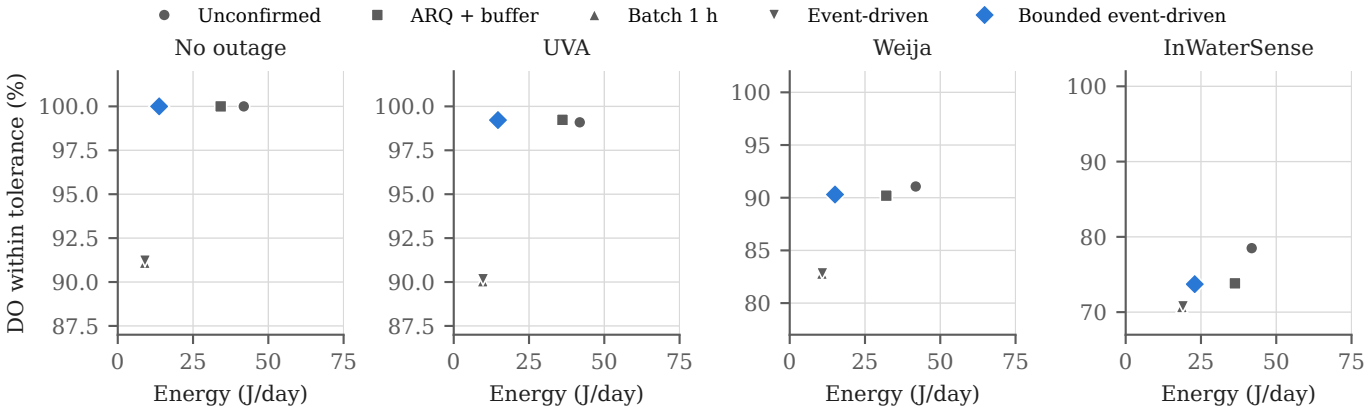


Fig. 4. Energy against the real-time picture of DO (share of slots within the L2 tolerance), LoRaWAN SF7, median over sites, one panel per link. Batch 1 h and event-driven batching nearly coincide. The bounded design reaches the picture of per-reading reporting at a fraction of its energy. Note the different vertical scales.

TABLE VI
ALARM LATENCY WITH THE LINK UP (MEDIAN OVER SITES OF EACH SITE'S MEDIAN, HOURS) AND TRANSMISSION ENERGY (MEDIAN OVER SITES, J/DAY); LORAWAN SF7. ENERGIES OF THE EVENT-DRIVEN DESIGNS ARE AVERAGED OVER THE FOUR EVENT SETTINGS; THE OTHER STRATEGIES SEND THE SAME FRAMES WHATEVER THE READINGS. ARQ: ARQ + BUFFER.

	Alarm latency (h)				Energy (J/day)			
Link	ARQ	Batch	Event	Bounded	ARQ	Batch	Event	Bounded
No outage	1.00	1.25	1.00	1.00	34.2	9.0	9.0	13.7
UVA	1.00	1.31	1.00	1.00	36.1	9.6	9.7	14.7
Weija	1.00	1.50	1.00	1.00	32.1	10.7	10.8	15.0
InWaterSense	1.15	1.50	1.00	1.00	36.3	18.7	19.0	22.9

be a placeholder for a sensor gap rather than the newest valid reading; and the node's copy of the server counted readings as known as soon as their upload began, although at 5-min sampling an SF12 upload spans several slots. Checking the fix revealed that the second defect could also affect any site after an outage, when a long backlog upload spans many slots. Both defects were fixed (deviation D4) and all sites were rerun. After the fix, newest-first packing is within tolerance at every scored slot of the no-outage link, on every site, parameter and radio, at 0.994–1.002 times the energy of FIFO packing; this is reported as a post-hoc result, and the pre-specified verdict stands. The lesson is general: a node that mirrors its server must model when data arrive, not only whether.

The tolerance level sets the price of the bound (Table IX; clean series, so the L2 energies differ slightly from the event-averaged values above). The picture stays within 1 percentage point of per-reading reporting at every level (H9 passes, 96/96 at L1 and L3; differences -0.6 to $+0.4$ pp at L1 and -0.1 to $+0.3$ pp at L3). The bound costs more at tighter tolerances: the trigger multiplies the plain design's energy by 1.18×–2.24× at L1, 1.06×–1.51× at L2 and only 1.01×–1.10× at L3. Yet even at L1 the bounded design is 1.18×–1.48× cheaper than per-reading ARQ (H10 passes, 24/24), and at L3 1.39×–3.35×.

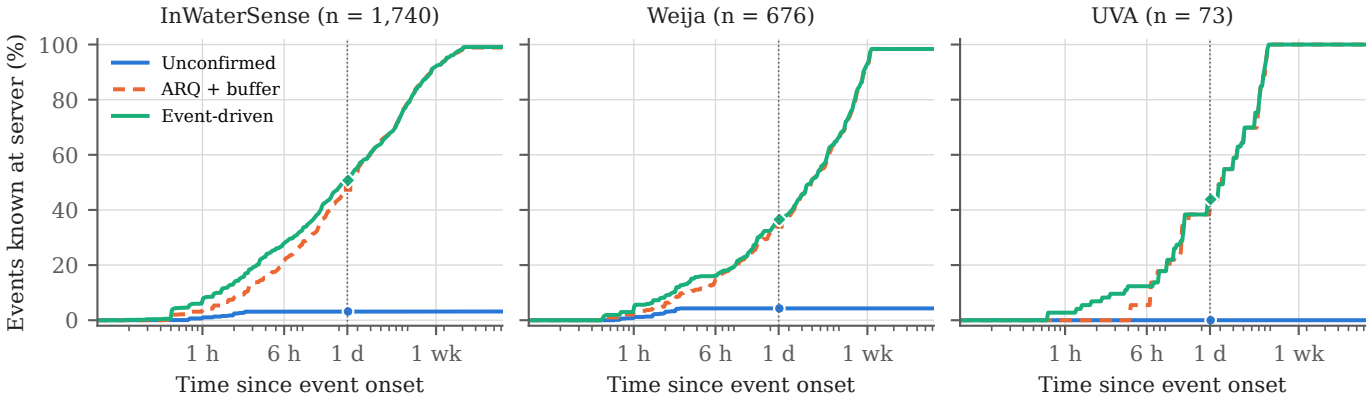


Fig. 5. Detected events whose onset fell in an outage: share known at the server by the time on the horizontal axis (LoRaWAN SF7, pooled over sites; markers at 1 day). Unconfirmed reporting learns of almost none; the buffering strategies learn of nearly all, after the outage.

TABLE VII
REAL-TIME PICTURE: SHARE OF SCORED SLOTS WITHIN THE L2 TOLERANCE (MEDIAN OVER SITES; LORAWAN SF7; SERIES WITHOUT INJECTED EVENTS). ARQ: ARQ + BUFFER.

Link	Param.	Unconf.	ARQ	Batch	Event	Bounded
No outage	DO	100.0%	100.0%	91.1%	91.2%	100.0%
	pH	100.0%	100.0%	92.1%	92.1%	100.0%
	T	100.0%	100.0%	96.8%	96.9%	100.0%
	EC	100.0%	100.0%	98.3%	98.3%	100.0%
UVA	DO	99.1%	99.2%	90.0%	90.2%	99.2%
	pH	99.2%	99.5%	91.8%	91.9%	99.5%
	T	99.0%	99.2%	95.6%	95.7%	99.2%
	EC	99.3%	99.5%	97.8%	97.8%	99.5%
Weija	DO	91.1%	90.2%	82.8%	82.8%	90.3%
	pH	93.7%	93.0%	87.0%	87.1%	93.1%
	T	88.6%	87.7%	84.8%	84.8%	87.5%
	EC	90.6%	89.8%	88.1%	88.1%	89.9%
InWaterSense	DO	78.5%	73.8%	70.7%	70.8%	73.7%
	pH	84.1%	79.9%	76.0%	76.6%	79.5%
	T	72.7%	66.9%	65.6%	65.6%	67.3%
	EC	80.5%	71.5%	71.1%	71.1%	72.1%

TABLE VIII
DETECTED EVENTS WHOSE ONSET FELL IN AN OUTAGE (LORAWAN SF7, POOLED OVER SITES): NUMBER OF EVENTS, SHARE THE SERVER EVER LEARNS OF, AND MEDIAN LATENCY OF THOSE IT LEARNS OF (HOURS).

		Known ever			Median latency (h)			
Link	n	Unconf.	ARQ	Event	ARQ	Batch	Event	Bounded
InWaterSense	1,740	3.2%	98.9%	99.1%	25.5	24.6	21.2	22.9
Weija	676	4.3%	98.4%	98.4%	48.3	44.3	44.3	48.3
UVA	73	0.0%	100.0%	100.0%	31.4	31.9	31.9	31.9

Without the trigger, tight tolerances are not met: at L1 the plain design's picture falls short of per-reading reporting's by a median of -26.5 percentage points for DO, -19.3 for pH, -6.3 for EC and -5.5 for T.

The answer to RQ4 is yes, with the link up. Bounded event-driven batching delivers every reading, raises alarms as fast as per-reading reporting, keeps the server's picture within a chosen tolerance as often as per-reading reporting does, and costs $1.07\times$–$1.55\times$ the energy of plain hourly batching, while per-reading ARQ uses $1.30\times$–$2.32\times$ as much energy at the reference tolerance. What no scheme can fix is the outage itself: events and data during an outage reach the server only when the link returns.

TABLE IX
ENERGY (MEDIAN J/DAY OVER SITES, LORAWAN SF7, SERIES WITHOUT INJECTED EVENTS) AT THREE TOLERANCE LEVELS. EVENT: WITHOUT THE TRIGGER (TOLERANCE-INDEPENDENT). ARQ / BOUNDED: MEDIAN OVER SITES OF EACH SITE'S RATIO, SO IT DIFFERS FROM THE RATIO OF THE MEDIAN ENERGIES.

Level	Link	ARQ + buffer	Event	Bounded	ARQ / Bounded
L1 (tight)	No outage	34.2	9.0	19.9	1.48×
	InWaterSense	36.3	18.9	28.3	1.23×
L2	No outage	34.2	9.0	13.5	2.33×
	InWaterSense	36.3	18.9	22.7	1.55×
L3 (loose)	No outage	34.2	9.0	9.9	3.28×
	InWaterSense	36.3	18.9	19.8	1.82×

VIII. DISCUSSION

A. Practical guidance

The results suggest five rules for battery-powered water-quality telemetry on LPWANs.

(1) *Know which regime loses your data.* Short radio losses and long system outages call for different remedies, and in the two water-quality deployments studied, the long outages held most of the lost data. A delivery log with frame counters, and a note of when the gateway and server were down, is enough to tell them apart (Sec. IV). Simulations and estimators that assume memoryless loss should be checked against such a log.

(2) *Do not fight outages with retransmissions.* ARQ is useful against radio losses only. Under outages, confirm, buffer and back off; on NB-IoT, back off even if nothing is buffered, because searching for a cell in every slot of an outage cost up to $6.53\times$ the energy of backing off in the swept range (Sec. V).

(3) *Batch routine data, and make the batch leave early when it matters.* Hourly batching delivers every reading for a fraction of per-reading energy. Two triggers close its two gaps: an event alarm, for timeliness, and a bounded-error trigger that mirrors the server's estimate, for the real-time picture. The tolerance is then the operator's energy knob (Sec. VII). At a sampling interval of an hour or more there is little left to batch.

(4) *Use the site's own diel pattern as the interim estimate, and judge reconstruction against it.* Between deliveries, a site's own day–night pattern is the best simple estimator for DO and pH; neighbours help temperature, and help most when bridging a gap after the data return. A reconstruction method evaluated only against the last value will overstate what spatial information adds (Sec. VI).

(5) *Engineer the outages themselves.* No reporting scheme shortens an outage; events that start during one reach the server only when it ends. Gateway diversity masked a month-long gateway outage at UVA without losing a frame (Sec. IV-C), and deployment reports show that watchdog resets bound how long silent gateway failures last [6], [7]. How long system outages last is partly an operations variable.

B. Validity and limitations

Outage records. The outage evidence comes from two water-quality deployments and two LoRaWAN networks. The Weija record has few long outages and serves as qualitative support only, and its unflagged interpolation hides some short outages. Neither water-quality record separates a network outage from a node that was switched off or out of power, so "delivered" after buffering is an upper bound for those periods; replaying every missing slot as a down link is pessimistic for ARQ, which is why the radio losses that ARQ can recover are also added. UVA's "system" outages include the operators' own logging pipeline, which from a user's point of view is still data that did not arrive.

Energy. The energy model rests on published measurements of specific modules and network settings, with an ideal battery. The energy an NB-IoT modem spends searching when there is no network has no published measurement that could be found, so it was swept. The duty-cycle wait of LoRaWAN retransmissions is not modelled; it changes delay, not energy. Ratios between strategies are therefore more reliable than absolute lifetimes.

What the model leaves out. Draining a large backlog can starve live traffic [6]; a drain rate limit or interleaving of live readings with the backlog would avoid it, at some cost in delay. Buffers are modelled as unlimited and perfect, whereas real storage fails [7], [27]. Clock faults and reordering are not simulated.

Events and detection. The events are synthetic steps and ramps injected into real series, and the detector, whose parameters were fixed in advance, catches slow ramps poorly (14% of 5σ ramps). The alarm comparisons concern how fast the server learns of a detected event, not how many events are detected.

Rivers and tolerances. The river data cover one year in one country, and some lower-Delaware sites are tidal, where conductivity follows the tide rather than the day. The tolerances are engineering choices, not regulatory limits; the sensitivity analysis shows how the results change with them. The server's estimator is deliberately simple, and a learned estimator could shorten the bounded design's extra uploads, but not the gap a long outage leaves.

Pre-specification. The hypotheses and decision rules were written down before the runs and kept with the code, but not registered publicly. Every verdict is reported, including the two failures, and every deviation with its reason and its effect.

C. Conclusion

Real deployments lose data in two regimes: short radio losses, and rare system outages that, in the two water-quality deployments studied, hold most of the lost data. Remedies designed for short, memoryless losses fit the first regime and not the second. Retransmission spends energy on outages it cannot bridge, and reconstruction, whether from a site's own history or from its neighbours, cannot keep day-long gaps within tolerance. Buffering with back-off can, at an energy within about a quarter of that of not recovering at all, and hourly batching makes it cheaper still. A reporting design that batches hourly, sends alarms at once, and leaves early whenever the server's estimate would leave tolerance delivers every reading, matches per-reading reporting on alarm latency and on the real-time picture whenever the link is up, and needs only a fraction of the energy of per-reading ARQ, which uses $1.30\times$–$2.32\times$ as much. The remaining gap is the outage itself, which only network engineering, such as gateway diversity and watchdogs, can shorten. Future work includes longer and tropical outage records, measuring NB-IoT search energy without a network, modelling backfill contention and storage failure, and testing the design on hardware.

Data and Code Availability

The USGS [42], InWaterSense [35], LoED [37] and UVA [36] data are public. The Weija records were provided by the deployment's operators [2]. The code, configurations, seeds and scripts that regenerate every number, table and figure in this paper, with pinned software versions, will be archived at Zenodo on acceptance.

Acknowledgment

This work received no external funding.

References

[1] K. S. Adu-Manu, C. Tapparello, W. Heinzelman, F. A. Katsriku, and J.-D. Abdulai, "Water quality monitoring using wireless sensor networks: Current trends and future research directions," *ACM Transactions on Sensor Networks*, vol. 13, no. 1, pp. 1–41, 2017.

[2] K. S. Adu-Manu, F. A. Katsriku, J.-D. Abdulai, and F. Engmann, "Smart river monitoring using wireless sensor networks," *Wireless Communications and Mobile Computing*, vol. 2020, pp. 1–19, 2020, art. no. 8897126.

[3] F. Ahmedi and L. Ahmedi, "Dataset on water quality monitoring from a wireless sensor network in a river in Kosovo," *Data in Brief*, vol. 44, p. 108486, 2022.

[4] T. Le Dinh, W. Hu, P. Sikka, P. Corke, L. Overs, and S. Brosnan, "Design and deployment of a remote robust sensor network: Experiences from an outdoor water quality monitoring network," in *Proc. 32nd IEEE Conf. Local Computer Networks (LCN)*. IEEE, 2007, pp. 799–806.

[5] K. Langendoen, A. Baggio, and O. Visser, "Murphy loves potatoes: Experiences from a pilot sensor network deployment in precision agriculture," in *Proc. 20th IEEE Int. Parallel and Distributed Processing Symp. (IPDPS)*. IEEE, 2006, p. 8 pp.

[6] F. Ingelrest, G. Barrenetxea, G. Schaefer, M. Vetterli, O. Couach, and M. Parlange, "SensorScope: Application-specific sensor network for environmental monitoring," *ACM Transactions on Sensor Networks*, vol. 6, no. 2, pp. 17:1–17:32, 2010.

[7] M. Brzozowski, M. Frohberg, and P. Langendoerfer, "Real-world deployments of sensor networks: Practical lessons for researchers," in *Wired/Wireless Internet Communications (WWIC 2018), LNCS vol. 11618*. Springer International Publishing, 2018, pp. 42–53.

[8] P. P. Puluckul and M. Weyn, "Experiences with sub-arctic sensor network deployment," in *Proc. IEEE Wireless Communications and Networking Conf. (WCNC)*. IEEE, 2025, pp. 1–6.

[9] M. Zennaro, E. Pietrosemoli, A. Soltobaev, T. Sultanov, and B. Moldobekov, "From mountains to data: low-cost weather stations in Kyrgyzstan's challenging terrain," *Frontiers in Communications and Networks*, vol. 6, 2025, art. no. 1505375.

[10] B. Sinopoli, L. Schenato, M. Franceschetti, K. Poolla, M. Jordan, and S. Sastry, "Kalman filtering with intermittent observations," *IEEE Transactions on Automatic Control*, vol. 49, no. 9, pp. 1453–1464, 2004.

[11] K. Huang, W. Liu, Y. Li, and B. Vucetic, "To retransmit or not: Real-time remote estimation in wireless networked control," in *Proc. IEEE Int. Conf. Communications (ICC)*. IEEE, 2019, pp. 1–7.

[12] F. Nadeem, Y. Li, B. Vucetic, and M. Shirvanimoghaddam, "HARQ optimization for real-time remote estimation in wireless networked control," arXiv:2201.05838v3, 2023.

[13] L. Kong, M. Xia, X.-Y. Liu, G. Chen, Y. Gu, M.-Y. Wu, and X. Liu, "Data loss and reconstruction in wireless sensor networks," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 11, pp. 2818–2828, 2014.

[14] A. Cini, I. Marisca, and C. Alippi, "Filling the G_ap_s: Multivariate time series imputation by graph neural networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022, arXiv:2108.00298.

[15] I. Marisca, A. Cini, and C. Alippi, "Learning to reconstruct missing data from spatiotemporal graphs with sparse observations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[16] Q. Chang, D. Tao, J. Wang, and R. Gao, "Deep compressed sensing based data imputation for urban environmental monitoring," *ACM Transactions on Sensor Networks*, vol. 20, no. 1, pp. 1–21, 2023, art. no. 17.

[17] J. Zhao and R. Govindan, "Understanding packet delivery performance in dense wireless sensor networks," in *Proc. 1st Int. Conf. Embedded Networked Sensor Systems (SenSys)*. ACM, 2003, pp. 1–13.

[18] L. Bhatia, M. Breza, R. Marfievici, and J. A. McCann, "LoED: The LoRaWAN at the edge dataset," in *Proc. 3rd Workshop on Data: Acquisition To Analysis (DATA '20)*. ACM, 2020, pp. 7–8.

[19] F. Nikseresht, V. A. L. Sobral, J. L. Goodall, and B. Campbell, "Dataset: Long-term LoRaWAN communication metadata from an urban deployment," in *Proc. 12th ACM Int. Conf. Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '25)*. ACM, 2025, pp. 412–417.

[20] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, and S. Ulukus, "Age of information: An introduction and survey," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 5, pp. 1183–1210, 2021.

[21] B. Sun and N. Pappas, "Resource-constrained semantic-aware remote estimation with overlapping sensor coverage," in *Proc. ACM MobiHoc*, 2026, to appear; arXiv:2609.07563.

[22] C. Luo, F. Wu, J. Sun, and C. W. Chen, "Compressive data gathering for large-scale wireless sensor networks," in *Proc. 15th Annu. Int. Conf. Mobile Computing and Networking (MobiCom)*. ACM, 2009, pp. 145–156.

[23] J. Cheng, Q. Ye, H. Jiang, D. Wang, and C. Wang, "STCDG: An efficient data gathering algorithm based on matrix completion for wireless sensor networks," *IEEE Transactions on Wireless Communications*, vol. 12, no. 2, pp. 850–861, 2013.

[24] X. Piao, Y. Hu, Y. Sun, B. Yin, and J. Gao, "Correlated spatio-temporal data collection in wireless sensor networks based on low rank matrix approximation and optimized node sampling," *Sensors*, vol. 14, no. 12, pp. 23 137–23 158, 2014.

[25] E. E. Peterson and J. M. Ver Hoef, "A mixed-model moving-average approach to geostatistical modeling in stream networks," *Ecology*, vol. 91, no. 3, pp. 644–651, 2010.

[26] P. McDonald, D. Geraghty, I. Humphreys, S. Farrell, and V. Cahill, "Sensor network with delay tolerance (SeNDT)," in *2007 16th International Conference on Computer Communications and Networks*. IEEE, 2007, pp. 1333–1338.

[27] M. J. Murphy, Ø. Tveito, E. F. Kleiven, I. Rais, E. M. Soininen, J. M. Bjørndalen, and O. Anshus, "Experiences building and deploying wireless sensor nodes for the arctic tundra," in *Proc. IEEE/ACM 21st Int. Symp. Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 2021, pp. 376–385.

[28] L. Casals, B. Mir, R. Vidal, and C. Gomez, "Modeling the energy performance of LoRaWAN," *Sensors*, vol. 17, no. 10, p. 2364, 2017.

[29] F. Michelinakis, A. S. Al-Selwi, M. Capuzzo, A. Zanella, K. Mahmood, and A. Elmokashfi, "Dissecting energy consumption of NB-IoT devices empirically," *IEEE Internet of Things Journal*, vol. 8, no. 2, pp. 1224–1242, 2021.

[30] A. Sørensen, H. Wang, M. J. Remy, N. Kjettrup, R. B. Sørensen, J. J. Nielsen, P. Popovski, and G. C. Madueño, "Modeling and experimental validation for battery lifetime estimation in NB-IoT and LTE-M," *IEEE Internet of Things Journal*, vol. 9, no. 12, pp. 9804–9819, 2022.

[31] A. Azari, C. Stefanovic, P. Popovski, and C. Cavdar, "On the latency-energy performance of NB-IoT systems in providing wide-area IoT connectivity," *IEEE Transactions on Green Communications and Networking*, vol. 4, no. 1, pp. 57–68, 2020.

[32] S. Ugwuanyi, G. Paul, and J. Irvine, "Survey of IoT for developing countries: Performance analysis of LoRaWAN and cellular NB-IoT networks," *Electronics*, vol. 10, no. 18, p. 2224, 2021, art. no. 2224.

[33] G. M. Dias, B. Bellalta, and S. Oechsner, "A survey about prediction-based data reduction in wireless sensor networks," *ACM Computing Surveys*, vol. 49, no. 3, pp. 1–35, 2016.

[34] S. Santini and K. Römer, "An adaptive strategy for quality-based data reduction in wireless sensor networks," in *Proc. 3rd Int. Conf. Networked Sensing Systems (INSS)*, 2006.

[35] L. Ahmedi, "InWaterSense dataset: Data from a wireless sensor network on water quality monitoring in a river in Kosovo," Mendeley Data, V1, 2021.

[36] F. Nikseresht, V. A. L. Sobral, J. L. Goodall, and B. Campbell, "Long-term LoRaWAN communication metadata from an urban deployment," University of Virginia Dataverse (LibraData), 2025.

[37] L. Bhatia, M. Breza, R. Marfievici, and J. A. McCann, "LoED: The LoRaWAN at the edge dataset," Zenodo, 2020.

[38] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[39] H. Akaike, "A new look at the statistical model identification," *IEEE Transactions on Automatic Control*, vol. 19, no. 6, pp. 716–723, 1974.

[40] *Waspmote Plug & Sense! Technical Guide (product line v15)*, Libelium Comunicaciones Distribuidas, 2020, document version v9.2.

[41] LoRa Alliance Technical Committee Regional Parameters Workgroup, "RP002-1.0.3 LoRaWAN regional parameters," LoRa Alliance, Tech. Rep., 2021.

[42] U.S. Geological Survey, "USGS water data for the nation (National Water Information System)," 2024, instantaneous values for 2023, accessed Oct. 2026.

[43] B. B. Benson and D. Krause, "The concentration and isotopic fractionation of oxygen dissolved in freshwater and seawater in equilibrium with the atmosphere," *Limnology and Oceanography*, vol. 29, no. 3, pp. 620–632, 1984.

[44] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, p. 80, 1945.

[45] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1–2, pp. 100–115, 1954.